



Chapter 1

Fundamental Knowledge





Outline

- ★ **Concepts of Digital Systems;**
- ★ **Number System and Conversion;**
- ★ **Signed Binary Number;**
- ★ **Binary Codes**





1.1 Overview

1.1.1 Digital system

I. Information and digits

We are now in an era of information ! Do we think about: **What is information?**

Definition of Information: The information can be defined from many perspectives; such as: " information is the quantity that represents the numerical characteristics of physical quantities. " "Information is the reflection of matters." "Information is the basis for human communication." ...,

Generally speaking, “Information is the description of all kinds of differences existing in the objective world.”





II. Digital systems

What is a digital system?

A digital system is an entity that can store, transmit, and process digital signals. It is formed by interconnecting digital logic circuits that implement various functions. For example, digital computers.

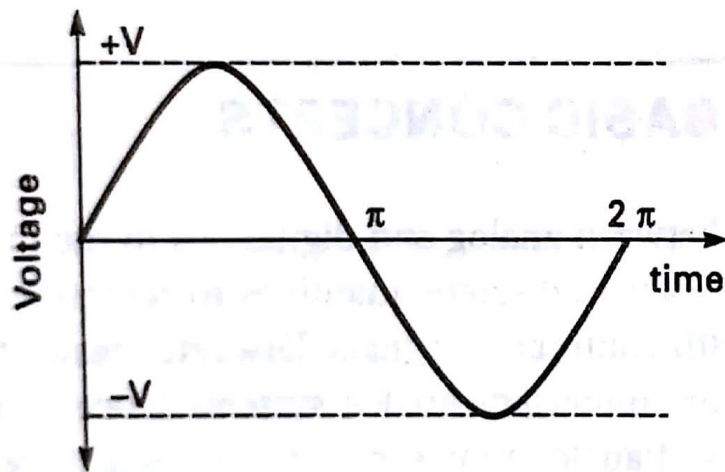
1. Digital signal

If the signal change is discrete in time and value, it can be called the discrete signal. The change of the discrete signal can be reflected by different numbers, so it is also called a digital signal.

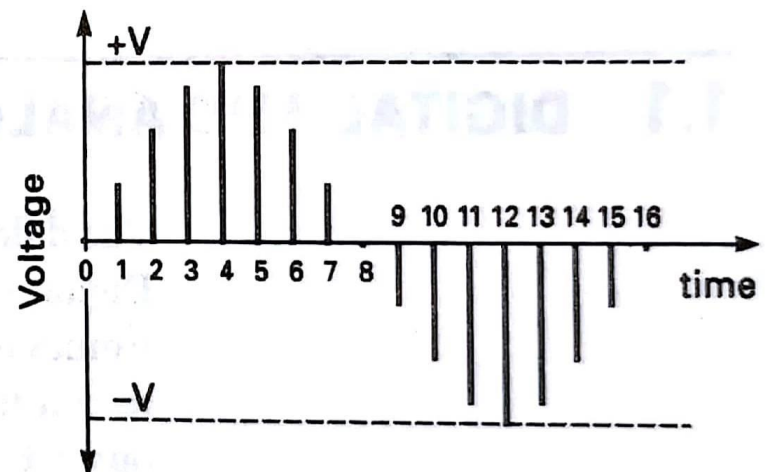
E.g. student scores, factory product statistics, circuit switch status, etc.



Analog vs Discrete



(a) Analog representation



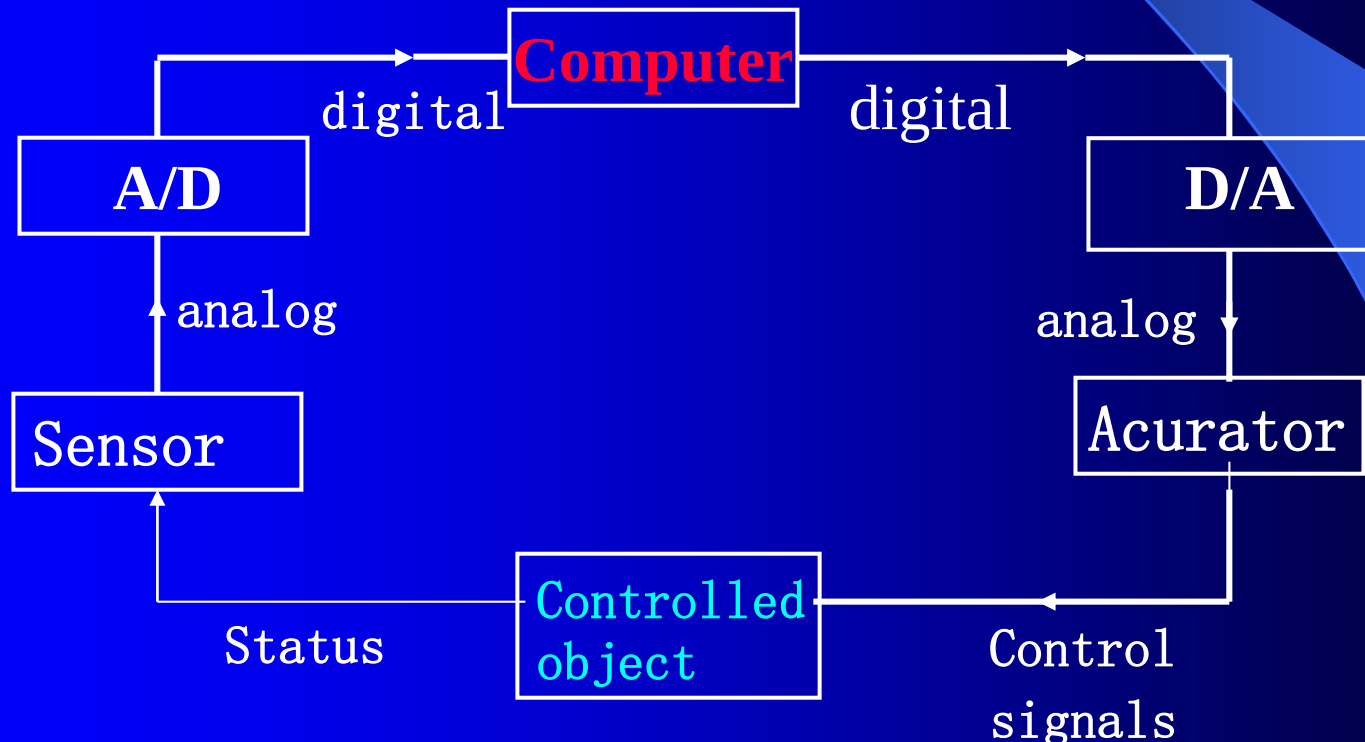
(b) Discrete representation





Digital systems deal with digital signals. If a digital system needs to communicate with analog signals, it must have analog/digital (A/D) conversion and digital/analog (D/A) conversion circuits to convert the signal types.

E.g. a typical computer control system:





2. Digital logic circuit

Electronic circuits used to process digital signals are called digital circuits. Since various functions of the digital circuit are implemented by logical operations and logic judgments, digital circuit is also called digital logic circuit or just logic circuit.

Features:

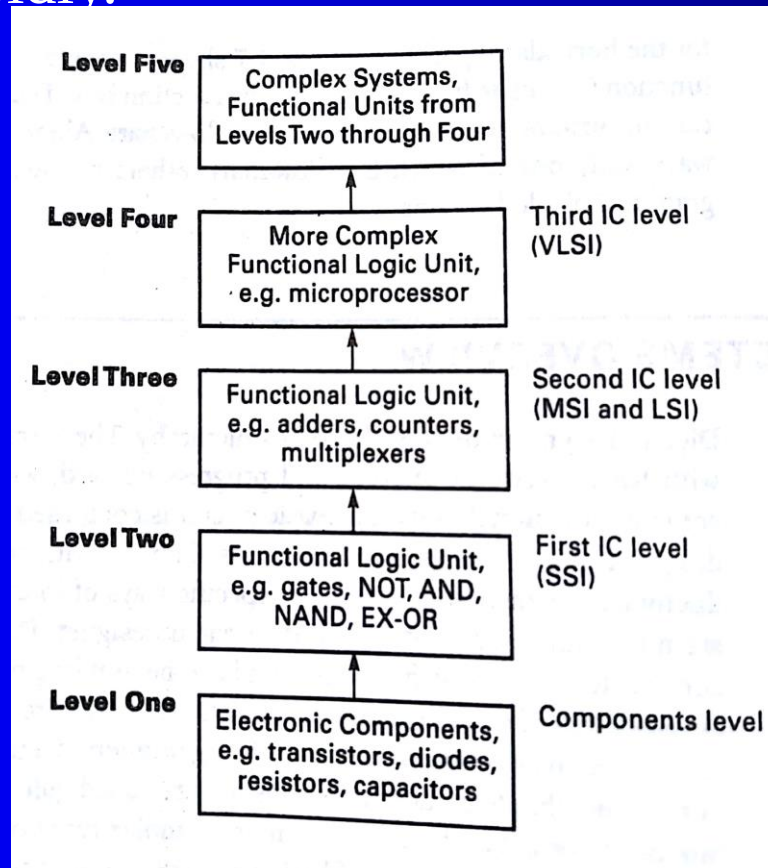
- (1) The basic signal of the circuit is a binary signal. It is represented by two stable physical states such as "high" or "low" of voltage in a circuit, "on" or "off" of a switch, "on" or "off" of a transistor.
- (2) Semiconductor devices in circuits generally operate in the on and off states.
- (3) Simple circuit, low power consumption, and easy for integrated or large scale manufacturing; low product cost, convenient to use, and good versatility.
- (4) The digital system formed by the digital logic circuit has the advantages of high speed, high precision, strong functions, and good reliability.





Since the digital logic circuit has the above characteristics, the application of the digital logic circuit is very extensive.

With the development of semiconductor technology, digital integrated circuits have emerged and integrated circuits have developed rapidly.





III、 Computer

1. Digital computer

A digital computer is a digital system that can perform numerical calculation, data processing, control, and management functions automatically, quickly, and accurately.

2. Development of computer

Since the first digital computer was invented in 1946, its development speed has been astonishing. According to the composition of the main components of the computer, it has gone through four generations so far.

The general trend of computer development is: **speed**↑, **capability**↑, **size**↓, **cost**↓, **power consume**↓.





1.1.2 Digital logic circuit types and study methods

I. Digital logic circuit types

According to whether a circuit has a memory function, the digital logic circuit can be divided into two types: combinational logic circuit and sequential logic circuit.

Combinational Logic : If the stable output of a logic at any moment depends only on the input at that moment, and is independent of the circuit's past input, it is called a combinatorial logic circuit.

Since the output of this type of circuit is independent of the input signal of the past, no memory function is required.





Sequential Logic Circuit: If the stable output of a logic circuit at any moment depends not only on the input at that moment but also on the input in the past, it is called a sequential logic circuit.

Based on whether a circuit includes a unified clock, a sequential logic circuit can be further regarded as a **synchronous sequential** logic circuit and an **asynchronous sequential** logic circuit.





II. Study methods

Two major tasks in the study of logic circuits in digital systems: (1) analysis and (2) design.

(1) For an existing digital logic circuit, studying its performance and logic functions is called **logic circuit analysis**.

(2) Given the logic function, then design a logic circuit that constructs a predetermined function under a given condition is called **logic circuit design**.

The methods of logic circuit analysis and design are constantly changing with the rapid development of integrated circuits. The most mature method is the traditional method.





Traditional method of logic circuit analysis and design

Traditional Method: The traditional method is based on small-scale integrated circuits. It uses technical and economic indicators as the main performance indicators to evaluate the merits of a design scheme. The goal of designing is how to make a circuit the **simplest**.

How to achieve the simplest structure? When combining logic circuit designs, minimize the number of logic gates and connections in the circuit. In the design of sequential logic circuits, the number of flip-flops, logic gates, and connections in the circuit is minimized as much as possible.

Note that: In practical applications, a simplest design is not the best solution.

The best plan should meet the requirements of comprehensive performance and practical application. Therefore, after finding a simplest structure to achieve predetermined function by traditional methods, we should adjust accordingly according to the actual situation.



2. Design of MSI and LSI

- ▲ Fully understand the logic structure and external characteristics of various devices, so that a reasonable choice of devices;
- ▲ Fully use the capabilities of each selected device, complete the design of various types of circuits or functional modules with flexible methods;
- ▲ Minimize interconnections between chips.



3. Design method for programmable logic devices (PLD)

The emergence of various types of programmable logic devices has brought a completely new approach to logic design. We no longer use conventional hardwired methods to construct circuits, but use extensive computer software to program the devices to implement various logic functions, which brings great convenience to logic design.

4. Computer aided logic design method

In the face of increasingly complex integrated circuit chip design and digital system design, people have to use computer for auxiliary logic design more and more. At present, there are various softwares for designing digital systems that are sold on the market. Computer-aided logic design methods are constantly being promoted and applied. Many people believe that computer design automation has become an independent discipline in computer science.





1.2 Number Systems

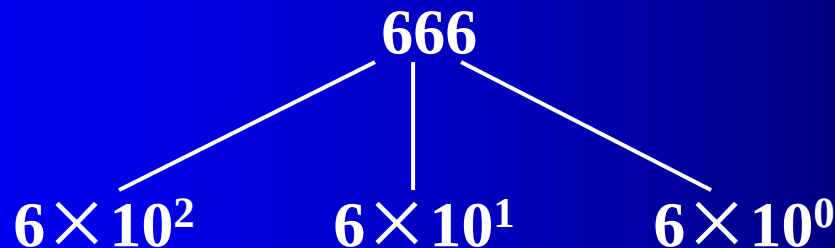
1.2.1 Carry system

The number system is a statistical rule that people count the number. We are familiar with base-10 number systems; the digital systems use base-2 number system.

一、Decimal numbers

The base 10 system contains 10 characters **0, 1, ..., 9**, **a carry is produced if a number is larger than 10**. When several numbers are used together to represent a number, the position of each character represent its value.

E.g.



The same character, 6, from left to right denotes 600, 60, 6, i.e.:

$$(666)_{10} = 6 \times 10^2 + 6 \times 10^1 + 6 \times 10^0$$





二、 Base-r number system

Generally speaking, a carry number system consists of two basic factors: base/radix and weight:

Base: the number of characters used in a number system. In base-r system, the characters are 0, 1, ..., R-1, the account number is r, Counting rule: a carry is produced if a number is larger than R.

Weight: in a number system, a constant used to indicate the magnitude of a value at a different position. Different digits have different weights.

The value of a position in a number is equal to the character multiplied by the digit weight.

The weight of the base-r number is an integer power of R

E.g., The weight of the decimal number is an integer power of 10, the first bit's weight is 10^0 , the second bit's weight is 10^1





Two representations of :

(1) Positional notation

$$(N)_R = (K_{n-1}K_{n-2}\dots K_1K_0 \cdot K_{-1}K_{-2}\dots K_{-m})_R$$

(2) Polynomial

$$\begin{aligned}(N)_R &= K_{n-1} \times R^{n-1} + K_{n-2} \times R^{n-2} + \dots + K_1 \times R^1 + K_0 \times R^0 \\ &\quad + K_{-1} \times R^{-1} + K_{-2} \times R^{-2} + \dots + K_{-m} \times R^{-m} \\ &= \sum_{i=-m}^{n-1} K_i R^i\end{aligned}$$

where: **R**—— Base, or radix ; **n**——integer part;

m—— fractional part;

K_i —— **a character in base R**, its value range is

$$0 \leq K_i \leq R-1 \quad (-m \leq i \leq n-1)$$





Summary of base r system:

(1) Digital numbers: $0 \sim R-1$;

(2) Counting rule: a carry is produced if a number is larger than R ;

(3) The weight of the base-r number is an integer power of R , the i th bit weight is R^i ($-m \leq i \leq n-1$).



III、 Binary numbers

The number of Base $R=2$ is binary system. Only two characters **0 and 1** exist in the binary character set, Counting rule: a carry is produced if a number is larger than 2, The weight of the binary number is an integer power of 2.

A binary number N can be represented as

$$\begin{aligned}
 (N)_2 &= (K_{n-1}K_{n-2}\dots K_1K_0.K_{-1}K_{-2}\dots K_{-m})_2 \\
 &= K_{n-1} \times 2^{n-1} + K_{n-2} \times 2^{n-2} + \dots + K_1 \times 2^1 + K_0 \times 2^0 \\
 &\quad + K_{-1} \times 2^{-1} + K_{-2} \times 2^{-2} + \dots + K_{-m} \times 2^{-m} \\
 &= \sum_{i=-m}^{n-1} K_i 2^i
 \end{aligned}$$

where: n —integer part; m —fractional part;

$$K_i \text{—} 0 \text{ or } 1, \quad -m \leq i \leq n-1$$





e.g. 1011.01 can be also be represented by:

$$(1011.01)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$$

Binary arithmetic :

Addition

$$0+0=0$$

$$0+1=1$$

$$1+0=1$$

$$1+1=0 \text{ (carry out 1)}$$

Subtraction

$$0-0=0$$

$$1-0=1$$

$$1-1=0$$

$$0-1=1 \text{ (borrow in 1)}$$

multiplication

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

Division

$$0 \div 1 = 0$$

$$1 \div 1 = 1$$





Binary system's advantages: Simple operation, easy physical implementation, convenient and reliable storage and transmission, and saving storage devices.

Since there are only two digital symbols 0 and 1 in the binary, one binary number can be represented by two different states of the electronic device. E.g., 1 and 0 may be represented by the off and on of the transistor, or can be represented by the high level and low level voltage. Therefore, the binary number is commonly used in digital systems.

Disadvantages: The number of digits is too long, making writing, memorizing, and reading very inconvenient.

Therefore, we often use octal numbers and hexadecimal numbers as an abbreviation for binary numbers when performing instruction writing, program input, and output.





IV、Octal system

The number of Base $R=8$ is octal system. 0, 1, ..., 7 exist in the octal character set, 8 in total, Counting rule: a carry is produced if a number is larger than 7, The weight of the octal number is an integer power of 8.

An octal number N can be represented as

$$\begin{aligned}
 (N)_8 &= (K_{n-1}K_{n-2}\dots K_1K_0 . K_{-1}K_{-2}\dots K_{-m})_8 \\
 &= K_{n-1} \times 8^{n-1} + K_{n-2} \times 8^{n-2} + \dots + K_1 \times 8^1 + K_0 \times 8^0 \\
 &\quad + K_{-1} \times 8^{-1} + K_{-2} \times 8^{-2} + \dots + K_{-m} \times 8^{-m} \\
 &= \sum_{i=-m}^{n-1} K_i 8^i
 \end{aligned}$$

where: n —integer part; m —fractional part;

K_i —a character within $0 \sim 7$, $-m \leq i \leq n-1$



V、Hexadecimal number (Hex)

The number of Base $R=16$ is hex system. 0, 1, ..., 9, A, B, C, D, E, and F exist in the hex character set, 16 in total, Counting rule: a carry is produced if a number is larger than 15, The weight of the octal number is an integer power of 16.

A hex number N can be represented as

$$\begin{aligned}
 (N)_{16} &= (K_{n-1}K_{n-2}\dots K_1K_0 . K_{-1}K_{-2}\dots K_{-m})_{16} \\
 &= K_{n-1} \times 16^{n-1} + K_{n-2} \times 16^{n-2} + \dots + K_1 \times 16^1 + K_0 \times 16^0 \\
 &\quad + K_{-1} \times 16^{-1} + K_{-2} \times 16^{-2} + \dots + K_{-m} \times 16^{-m} \\
 &= \sum_{i=-m}^{n-1} K_i 16^i
 \end{aligned}$$

where: n —integer part; m —fractional part; K_i —a character within 0~9、A~F, $-m \leq i \leq n-1$,





1.2.2 Number system conversion

Number system conversion means the conversion from a carry number into another. From the practical considerations, we must understand the conversion between binary numbers decimal numbers, and conversion between octal numbers and hexadecimal numbers.

I. Conversions between binary and decimal systems

1. Binary to decimal conversion

Method: Use Polynomial Representation

Expand the number into a power series with a base of 2; calculate each position's value and add the value to the total. **E.g.,** $(10110.101)_2$
 $= (?)_{10}$

$$\begin{aligned}(10110.101)_2 &= 1 \times 2^4 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^{-1} + 1 \times 2^{-3} \\ &= 16 + 4 + 2 + 0.5 + 0.125 \\ &= (22.625)_{10}\end{aligned}$$





2. Decimal to binary

Method: base division/multiplication

Steps: divide into integer and fraction, and covert separately.

Integer conversion: “Successive division” method ;

Fraction conversion: “Successive multiplication” method.

(1) Integer conversion

“Successive division” method , by repeated division of N by 2, the remainder of each division becomes the numeral in the new radix. The process is repeated until the least significant numeral is generated, (until the remainder is 0).



E. g. , $(35)_{10} = (?)_2$

	2	3	5		remainder	
2	1	7		1	(K_0)	LSB
2		8		1	(K_1)	
2		4		0	(K_2)	
2		2		0	(K_3)	
2	1			0	(K_4)	
	0			1	(K_5)	MSB

$$(35)_{10} = (100011)_2$$



(2) Fraction conversion

“Successive multiplication” method, N is multiplied by 2, producing a product consisting an integer and a fraction, the fraction is again multiplied by 2, producing a product, and so on until the fraction reaches 0; or until the number is carried out to sufficient digits.

e. g. , $(0.6875)_{10} = (?)_2$

	integer part	$\times$	0.6875	$\times 2$
MSB	$1(K_{-1}) \dots\dots$		1.3750	
		$\times$		2
	$0(K_{-2}) \dots\dots$		0.7500	
		$\times$		2
	$1(K_{-3}) \dots\dots$		1.5000	
		$\times$		2
LSB	$1(K_{-4}) \dots\dots$		1.0000	

:
 $(0.6875)_{10} = (0.1011)_2$





二、Hexadecimal and Octal to Binary Conversion

1. Binary to Octal

1. Binary to Octal: Partition the binary number into groups of three, starting at the decimal point and going left and right. If the bit is not enough, amend 0

E. g. , $(11100101.01)_2 = (?)_8$

<u>011</u>	<u>100</u>	<u>101</u>	.	<u>010</u>
↓	↓	↓		↓
3	4	5	.	2

$(11100101.01)_2 = (345.2)_8$





Octal to binary: just convert each octal bit to three binary number, and retain the decimal point position.

e. g. , $(56.7)_8 = (?)_2$

$$\begin{array}{ccc} 5 & 6 & . & 7 \\ \downarrow & \downarrow & & \downarrow \\ \underline{101} & \underline{110} & . & \underline{111} \end{array}$$

$$(56.7)_8 = (101110.111)_2$$





2. Conversion between Binary and Hex

Binary to Octal or Hex : Partition the binary number into groups of four, starting at the decimal point and going left and right. If the bit is not enough, amend 0.

$$\text{e. g. , } (101110.011)_2 = (?)_{16}$$

$$\begin{array}{ccc} \underline{0010} & \underline{1110} & . \quad \underline{0110} \\ \downarrow & \downarrow & \downarrow \\ 2 & E & . \quad 6 \end{array}$$

$$(101110.011)_2 = (2E.6)_{16}$$





Hex to binary: just convert each hex bit to three four number, and retain the decimal point position.

e. g. , $(5A.B)_{16} = (?)_2$

$$\begin{array}{ccc} 5 & A & . & B \\ \downarrow & \downarrow & & \downarrow \\ \underline{0101} & \underline{1010} & . & \underline{1011} \end{array}$$

$$(5A.B)_{16} = (1011010.1011)_2$$



Number system conversion

- Question: conversions between Hex and Octal



1.3 Signed number binary code

To mark the positive and negative of a number, we usually add a “+” in front of a number to indicate positive, add a “-” to indicate negative. In a digital system, symbols and values are represented by 0s and 1s. Generally, the most significant bit of a number is used as a sign bit, with 0 being positive and 1 being negative. Its format is

$$\begin{array}{ccccccc} X_f & X_{n-1} & X_{n-2} & \dots & X_1 & X_0 \\ \uparrow & & & & & \\ \text{sign bit} & & & & & \end{array}$$

The binary numbers with “+” and “-” are called as true values. The binary number represented by encoding a symbol and a numerical value is called a machine number or a machine code.

Common machine code contains Sign-magnitude code, 1s complement code, and 2s complement code





1.3.1 Sign-magnitude code

MSB represents sign (0 for positive, 1 for negative)

Magnitude bits represent the absolute value.

Sign+Magnitude

I、fractional number of Sign-magnitude code

a binary number $X = \pm 0.x_1x_2\dots x_m$, its sign-magnitude code is

$$[X]_{\text{sign-mag}} = \begin{cases} X & 0 \leq X < 1 \\ 1-X & -1 < X \leq 0 \end{cases}$$

{ sign bit { 0 positive
 1 negative
 { mag bit: unchange



E. g, if $X_1 = +0.1011$, $X_2 = -0.1011$

$$[X_1]_{\text{sign}} = 0.1011$$

$$[X_2]_{\text{sign}} = 1 - (-0.1011) = 1.1011$$

The fractional number, 0, in the sign-magnitude code is represented
0.0...0 or 1.0...0





II、 Integer number of Sign-magnitude code

A binary number $X = \pm x_{n-1}x_{n-2}\dots x_0$, its integer number of Sign-magnitude code is represented as:

$$[X]_{\text{sign}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^n - X & -2^n < X \leq 0 \end{cases}$$

e.g., , if $X_1 = +1101$, $X_2 = -1101$, thus:

$$[X_1]_{\text{sign}} = 01101$$

$$[X_2]_{\text{sign}} = 2^4 - (-1101) = 10000 + 1101 = 11101$$

The integer number, 0, in the sign-magnitude code is represented **0...0 or 1...0**





Advantages: Easy to understand and convert;

Disadvantages: difficult to calculate.

When adding or subtracting two numbers, it must use the sign of the two numbers of operands to determine whether to add or in the operation;

if it is subtraction, it must use the magnitude of the two numbers to determine the minuend and the subtrahend, and the symbol of the operation result. Obviously, this will increase the complexity of the operations.

To overcome the disadvantages, we have complement codes





1.3.2 1s complement code

Sign bit——MSB represents sign (0 for positive, 1 for negative) ;

Magnitude bit——Positive: same with sign-magnitude code; Negative , MSB is 1 (sign) . The rest bits are complemented bit-by-bit.

I、 Fractional part of 1s complement code

a binary number $X = \pm 0.x_1x_2\dots x_m$, its 1s complement code is

$$[X]_{1s} = \begin{cases} X & 0 \leq X < 1 \\ (2-2^{-m})+X & -1 < X \leq 0 \end{cases}$$



e. g. if $X_1 = +0.1011$, $X_2 = -0.1011$,
 $[X_1]_{1s} = 0.1011$

$$[X_2]_{1s} = 2 - 2^{-4} + X_2 = 10.0000 - 0.0001 - 0.1011 = 1.0100$$

-0	.	1	0	1	1
↓		↓	↓	↓	↓
1	.	0	1	0	0

The fractional number, 0, of 1s complement code is **0.0...0**
and 1.1...1



II、 Integer number of 1s complement code

If a number $X = \pm x_{n-1}x_{n-2}\dots x_0$, its complement code is

$$[X]_{1s} = \begin{cases} X & 0 \leq X < 2^n \\ (2^{n+1}-1)+X & -2^n < X \leq 0 \end{cases}$$

e. g. if $X_1 = +1001$, $X_2 = -1001$,

$$[X_1]_{1s} = 01001$$

$$\begin{aligned} [X_2]_{1s} &= (2^5-1)+X = (100000-1)+(-1001) \\ &= 11111-1001 = 10110 \end{aligned}$$

$$\begin{array}{ccccc} - & 1 & 0 & 0 & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 0 & 1 & 1 & 0 \end{array}$$

The Integer number, 0, in the 1s complement code is **00...0** and **11...1**



If using the 1s complement, both addition and subtractions can use the addition to calculate.

Addition and subtraction rules are as follows:

$$[X_1 + X_2]_{1s} = [X_1]_{1s} + [X_2]_{1s}$$

$$[X_1 - X_2]_{1s} = [X_1]_{1s} + [-X_2]_{1s}$$

During operation, the sign bit and the magnitude bit participate in the operation. When the sign bit has a carry, the carry should be added to the LSB of the result of the operation to get the final result.

1.3.3 2s complement code

Sign bit——MSB represents sign (0 for positive, 1 for negative) ;

Magnitude bit——Positive: same with sign-magnitude code; Negative, MSB is 1 (sign) . The rest bits are complemented bit-by-bit, add 1 to the complement.

I、 Fractional number of 2s complement code

If a fractional number $X = \pm 0.x_{-1}x_{-2}\dots x_{-m}$,

$$[X]_{2s} = \begin{cases} X & 0 \leq X < 1 \\ 2+X & -1 \leq X < 0 \end{cases}$$

e. g., is $X_1 = +0.1011$, $X_2 = -0.1011$,

$$[X_1]_{2s} = 0.1011$$

$$\begin{aligned} [X_2]_{2s} &= 2 + X = 10.0000 - 0.1011 \\ &= 1.0101 \end{aligned}$$

	-0	.	1	0	1	1
	↓		↓	↓	↓	↓
	1	.	0	1	0	0
+						1
	1	.	0	1	0	1

Note that, zero has a unique form: 0.0...0



II、 Integer number of 2s complement code

If a number $X = \pm x_{n-1}x_{n-2}\dots x_0$,

$$[X]_{2s} = \begin{cases} X & 0 \leq X < 2^n \\ 2^{n+1} + X & -2^n \leq X < 0 \end{cases}$$

E. g. , if $X_1 = +1010$, $X_2 = -1010$,

$[X_1]_{2s} = 01010$ (same with sign-magnitude code)

$[X_2]_{2s} = 2^5 + X = 100000 - 1010$

$= 10110$ (are complemented bit-by-bit,
add 1 to the complement)

Note that, zero has a unique form 00...0





If using the 2s complement, both addition and subtractions can use the addition to calculate

Addition and subtraction rules are as follows :

$$[X_1 + X_2]_{2s} = [X_1]_{2s} + [X_2]_{2s}$$

$$[X_1 - X_2]_{2s} = [X_1]_{2s} + [-X_2]_{2s}$$

During operation, the sign bit and the magnitude bit participate in the operation. When the sign bit has a carry, the carry must be dropped.





1.4 Common decimal code

1.4.1 Binary Coded Decimal code

4-bit binary code representing decimal numbers is referred to BCD(Binary Coded Decimal) .

BCD has the binary number's notation, and features of decimal number. Common BCD codes are **8421 code, 2421 code and excess-3 code.**





Characters 0~9 in decimal number, 8421 code、 2421 code and excess-3 code are summarized as:

Decimal number	8421code	2421code	excess-3code
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100





I、8421 Code

8421 code: It is a weighted code that represents a decimal character in 4 bit binary code, and the 4 bit binary code from high to low is 2^3 、 2^2 、 2^1 、 2^0 , **i.e., 8、4、2、1, so is called 8421 code.**

The 8421 codes of 0~9 are identical with the binary number. Therefore, 8421 yards is an intermediate form widely used in man-machine contact.

Note:

(1) In 8421 code, 1010 ~ 1111 are invalid(since no correspondent decimal characters);





1. Conversion between 8421 code and decimal numbers

A number with n decimal digits requires $4n$ bits in 8421 code; in other words, each group of four bits represent one decimal bit. E.g.

$$(258)_{10} = (0010\ 0101\ 1000)_{8421}$$

$$(0001\ 0010\ 0000\ 1000)_{8421} = (1208)_{10}$$

2. Differences with binary number

e.g.,

$$(28)_{10} = (11100)_2 = (00101000)_{8421}$$





II. 2421 code

2421 code: It is a weighted code that represents a decimal character in 4 bit binary code, and the 4 bit binary code from high to low is **2、4、2、1**, so is called **2421 code**.

If a decimal number X 's 2421 code is $a_3 a_2 a_1 a_0$, thus, the value is

$$X = 2a_3 + 4a_2 + 2a_1 + 1a_0$$

$$\text{e.g., } (1101)_{2421} = (7)_{10}$$

1. 2421 code to decimal number

The conversion is bit-by-bit,

$$(0010\ 0001\ 1110\ 1011)_{2421} = (2185)_{10}$$





1.4.2 Reliability code

Improve the reliability of the system

To reduce errors in data communication and processing, several coding methods are proposed.

I. Gray code

1. Only one single bit changes when going from one value to the next or previous one.

2. to reduce errors when data is changing





3. Convert binary number to Gray code.

Binary code $B = B_{n-1}B_{n-2} \dots B_{i+1}B_i \dots B_1B_0$

Objective Gray code $G = G_{n-1}G_{n-2} \dots G_{i+1}G_i \dots G_1G_0$

we have:
$$\begin{cases} G_{n-1} = B_{n-1} \\ G_i = B_{i+1} \oplus B_i \quad 0 \leq i \leq n-2 \end{cases}$$

where, “ $\oplus$ ” is XOR, whose arithmetic rules are:

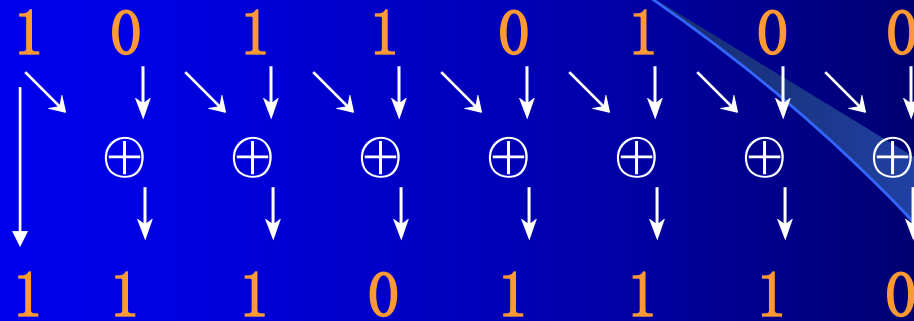
$$0 \oplus 0 = 0; \quad 0 \oplus 1 = 1;$$

$$1 \oplus 0 = 1; \quad 1 \oplus 1 = 0$$





e. g. ,



Binary code

Gray code

Excise: How to convert a Gray code to the binary code?





III. Parity-check code

Parity code is a code used to check whether a code has generated an error during transmission.

1. Component:

Two parts { **Information bit a group of binary number**
Parity bit—— only one bit

2. Coding methods: Two ways

Odd parity: to produce odd number of 1 in the code

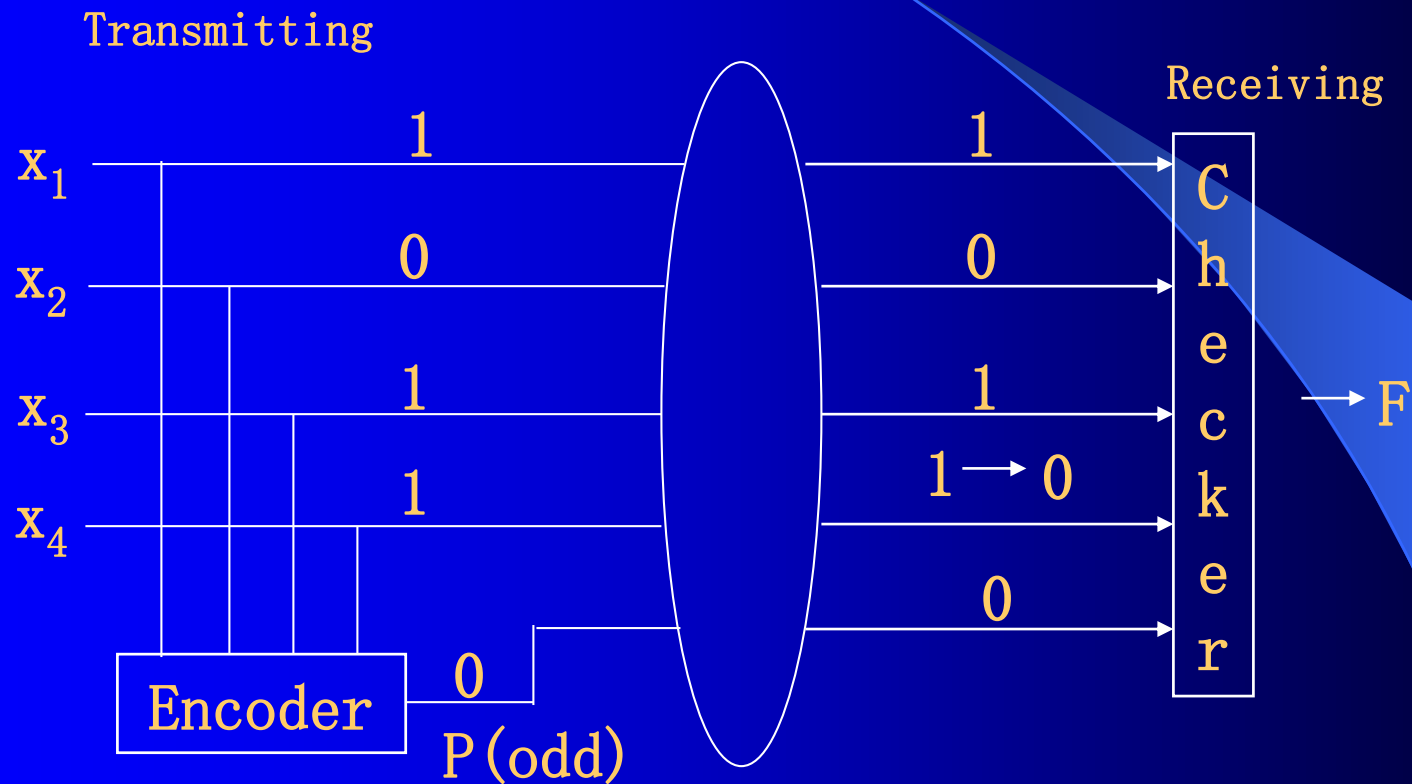
Even parity: to produce even number of 1 in the code

E. g. , Inf. bit (7 bit)	Odd parity (1 bit)	Even parity (1 bit)
1001100	0	1





3. The working principle of the parity code





4. Features

- (1) **Easy to implement;**
- (2) **Parity code has only error detection capability and no error correction capability.**
- (3) **can find single mistake, but cannot find two mistakes.**



Representing Text Data

- Use codes to represent characters.
- ASCII code (American Standard Code for Information Interchange)
 - 7-bit code, but usually manipulate as 8-bit.
 - Common ASCII codes:
Space —32 0~9—48~57
A~Z—65~90 a~z—97~112
- Unicode
 - Unicode 8, 16, and 32.